

COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithms	SECTION:	CPE21S1
DATE PERFORMED:	July 9, 2026	DATE SUBMITTED:	July 13, 2026
NAME:	Benj Federic C. De Leon	INSTRUCTOR:	Engr. Jimlord M. Quejado

ACTIVITY NO. 1.1 Basic C++ Programming

1. PROCEDURE OUTPUT

ILO A: Create Code That Follows the Basic C++ Code Structure

INPUT:

```
HOA_1 > sumoftwoNumbers.cpp > main()
1  #include <iostream>
2
3  //function prototype.
4  int sumoftwoNumbers(int firstNum, int secondNum);
5
6  int main(){
7
8      std::cout << "The sum of 24 and 7 is " << sumoftwoNumbers(24, 7);
9
10     return 0;
11 }
12
13 int sumoftwoNumbers(int firstNum, int secondNum){
14     int total = firstNum + secondNum;
15     return total;
16 }
```

OUTPUT:

```
PS C:\Users\Benj Federic De Leon\Desktop> g++ sumoftwoNumbers.cpp -o sumoftwoNumbers.exe
1\CPE 010-CPE21S1\" ; if ($?) { g++ sumoftwoNumbers.cpp -o sumoftwoNumbers.exe
The sum of 24 and 7 is 31
PS C:\Users\Benj Federic De Leon\Desktop>
```

INPUT:

```
CPE 010-CPE21S1 > checkinglargerNumber.cpp > main()
1  #include <iostream>
2
3  void checkNum(int a, int b);
4
5  int main(){
6
7
8      int num1;
9      int num2;
10
11     for(int i=0; i<1; i++){
12         std::cout << "Enter a number: ";
13         std::cin >> num1;
14
15         std::cout << "Enter a number: ";
16         std::cin >> num2;
17
18         checkNum(num1, num2);
19
20     }
21 }
22
23
24
25
26 void checkNum(int a, int b){
27     if (a < b){
28         std::cout << b << " is greater than " << a;
29     }
30     else if (a > b){
31         std::cout << a << " is greater than " << b;
32     }
33     else{
34         std::cout << "Both numbers are equal.";
35     }
36 }
37 }
```

OUTPUT:

```
PS C:\Users\Benj Federic De Leon\Documents\ic De Leon\Documents\CPE21S1\CPE 010-CPE2
checkinglargerNumber } ; if ($?) { .\che
Enter a number: 7
Enter a number: 24
24 is greater than 7
PS C:\Users\Benj Federic De Leon\Document
```

INPUT:

```

Welcome  main.cpp  bool.cpp  X  class_triangle.cpp  true.cpp
CPE 010-CPE21S1 > bool.cpp > ...
4
5  int main(){
6
7      bool a;
8      bool b;
9
10     std::cout << "Enter first boolean value(A): ";
11     std::cin >> a;
12
13     std::cout << "Enter second boolean value(B): ";
14     std::cin >> b;
15
16     display_logical_operators(a, b);
17
18     return 0;
19 }
20
21 bool display_logical_operators(bool a, bool b){
22
23     std::cout << "Logical AND: " << (a&&b);
24     std::cout << "\nLogical OR: " << (a||b);
25     std::cout << "\nLogical NOT(A): " << (!a);
26     std::cout << "\nLogical NOT(B): " << (!b);
27     std::cout << "\nLogical XOR: " << (a != b);
28
29     return true;
30
31 }

```

OUTPUT:

```

PS C:\Users\Benj Federic De Leon\PS C:\Users\Benj Federic De Leon\
1\CPE 010-CPE21S1\" ; if ($?) { g1\CPE 010-CPE21S1\" ; if ($?)
Enter first boolean value(A): 1 Enter first boolean value(A): 1
Enter second boolean value(B): 1 Enter second boolean value(B): 1
Logical AND: 1 Logical AND: 0
Logical OR: 1 Logical OR: 1
Logical NOT(A): 0 Logical NOT(A): 0
Logical NOT(B): 0 Logical NOT(B): 1
Logical XOR: 0 Logical XOR: 1
PS C:\Users\Benj Federic De Leon\PS C:\Users\Benj Federic De Leon\
1\CPE 010-CPE21S1\" ; if ($?) { g1\CPE 010-CPE21S1\" ; if ($?)
Enter first boolean value(A): 0 Enter first boolean value(A): 0
Enter second boolean value(B): 1 Enter second boolean value(B): 1
Logical AND: 0 Logical AND: 0
Logical OR: 0 Logical OR: 1
Logical NOT(A): 1 Logical NOT(A): 1
Logical NOT(B): 1 Logical NOT(B): 0
Logical XOR: 0 Logical XOR: 1
PS C:\Users\Benj Federic De Leon\

```

ILO B: Implement Appropriate Class Definition and Instances Based on Given Requirements

INPUT:

```
CPE 010-CPE2151 > G+ triangle_class.cpp > main()
1  #include <iostream>
2
3  //class name should follow PascalCase
4  class Triangle{
5  private:
6      double totalAngle, angleA, angleB, angleC;
7  public:
8      //constructor
9      Triangle(double A, double B, double C);
10
11     //methods
12     void setAngles(double A, double B, double C);
13
14     //validation
15     const bool validateTriangle();
16 };
17
18 int main(){
19     //driver code
20     Triangle set1(40, 30, 210);
21     if(set1.validateTriangle()){
22         std::cout << "The shape is a valid triangle" << std::endl;
23     }else{
24         std::cout << "The shape is NOT a valid triangle. \n";
25     }
26 }
27
28 return 0;
29 }
30
31 Triangle::Triangle(double A, double B, double C){
32     angleA = A;
33     angleB = B;
34     angleC = C;
35     totalAngle = A+B+C;
36 }
37
38 const bool Triangle::validateTriangle() {
39     return (totalAngle <= 180);
40 }
41
42
```

OUTPUT:

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL
PS C:\Users\Benj Federic De Leon\Documents\> g++ triangle_class.cpp && if ($?) { .\triangle_class.exe }
The shape is NOT a valid triangle.
PS C:\Users\Benj Federic De Leon\Documents\>
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve Different Problems using the C++ Programming Language

The supplementary activities are meant to gauge your ability in using C++. The problems below range from easy to intermediate to advanced problems. Note your difficulties after answering the problems below.

1. Create a C++ program to swap the two numbers in different variables.
2. Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.
3. Create a C++ program that has a function that will calculate the distance between two points.
4. Modify the code given in ILO B and add the following functions:
 - a. A function to compute for the area of a triangle
 - b. A function to compute for the perimeter of a triangle
 - c. A function that determines whether the triangle is acute-angled, obtuse-angled or 'others.'

1. Create a C++ program to swap the two numbers in different variables.

INPUT:

```
CPE 010-CPE21S1 > G+ swapping_two_numbers.cpp > main()
1  #include <iostream>
2
3  // 1. Create a C++ program to swap the two numbers in different variables.
4
5  void swap(int &a, int &b);
6
7  int main(){
8      int x = 14;
9      int y = 28;
10
11      std::cout << "Before swap: x = " << x << ", y = " << y << "\n";
12
13      swap(x, y);
14
15      std::cout << "After swap: x = " << x << ", y = " << y << "\n";
16
17      return 0;
18  }
19
20  void swap(int &a, int &b){
21      int temp = a;
22      a = b;
23      b = temp;
24  }
```

OUTPUT:

```
PS C:\Users\Benj Federic De Leon\Documents\
1\CPE 010-CPE21S1\ > if ($?) { g++ swapp
ers }
Before swap: x = 14, y = 28
After swap: x = 28, y = 14
PS C:\Users\Benj Federic De Leon\Documents\
```

2. Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.

INPUT:

```
CPE 010-CPE21S1 > g++ kelvin_to_fahrenheit.cpp > main()
1  #include <iostream>
2
3  //2.Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.
4  double kelvin_to_Fahrenheit(double K);
5
6  int main(){
7
8      double K;
9
10     std :: cout << "Enter Temperature (Kelvin): ";
11     std :: cin >> K;
12
13     kelvin_to_Fahrenheit(K);
14
15     return 0;
16 }
17
18 double kelvin_to_Fahrenheit(double K){
19
20     double fahrenheit = (K - 273.15) * 9/5 + 32;
21
22     std:: cout << K << " in Kelvin is equal to " << fahrenheit << " Fahrenheit.\n";
23
24     return fahrenheit;
25 }
```

OUTPUT:

```
1\CPE 010-CPE21S1\ > if ($?) { g++ kelvin_to_fah
heit }
Enter Temperature (Kelvin): 115.15
115.15 in Kelvin is equal to -252.4 Fahrenheit.
PS C:\Users\Benj Federic De Leon\Documents\CPE21S1
```

3. Create a C++ program that has a function that will calculate the distance between two points.

INPUT:

```
CPE 010-CPE21S1 > g++ distance.cpp > main()
1  #include <iostream>
2  #include <cmath>
3
4  //3.Create a C++ program that has a function that will calculate the distance between two points
5  double distance(double x1, double y1, double x2, double y2);
6
7  int main(){
8
9      std::cout << "The distance is: " << distance(5, 23, 9, 5) << "\n";
10
11     return 0;
12 }
13
14 double distance(double x1, double y1, double x2, double y2){
15     double dx = x2 - x1;
16     double dy = y2 - y1;
17
18     return sqrt(dx*dx + dy*dy);
19 }
```

OUTPUT:

```
PS C:\Users\Benj Federic De Leon\Documents\CPE
1\CPE 010-CPE21S1>" ; if ($?) { g++ distance.c
The distance is: 18.4391
PS C:\Users\Benj Federic De Leon\Documents\CPE
```

4. Modify the code given in ILO B and add the following functions:
 - a. A function to compute for the area of a triangle

```
double Triangle::computeArea() {
    double radA = angleA * M_PI / 180.0;
    double radB = angleB * M_PI / 180.0;
    double radC = angleC * M_PI / 180.0;

    double sideA = std::sin(radA);
    double sideB = std::sin(radB);

    return 0.5 * sideA * sideB * std::sin(radC);
}
```

- b. A function to compute for the perimeter of a triangle

```
double Triangle::computePerimeter() {  
    double radA = angleA * M_PI / 180.0;  
    double radB = angleB * M_PI / 180.0;  
    double radC = angleC * M_PI / 180.0;  
  
    double sideA = std::sin(radA);  
    double sideB = std::sin(radB);  
    double sideC = std::sin(radC);  
  
    return sideA + sideB + sideC;  
}
```

- c. A function that determines whether the triangle is acute-angled, obtuse-angled or 'others.'

```
std::string Triangle::determineAngleType() {  
    if (angleA > 90 || angleB > 90 || angleC > 90) {  
        return "obtuse-angled";  
    }  
    else if (angleA == 90 || angleB == 90 || angleC == 90) {  
        return "Others (Right-angled)";  
    }  
    else {  
        return "Acute-angled";  
    }  
}
```

INPUT:

```
CPE 010-CPE21S1 > class_triangle.cpp > determineAngleType()
1  #include <iostream>
2  #include <cmath>
3
4  //class name should follow PascalCase
5  class Triangle{
6      private:
7          double totalAngle, angleA, angleB, angleC;
8      public:
9          //constructor
10         Triangle(double A, double B, double C);
11
12         //methods
13         void setAngles(double A, double B, double C);
14
15         //validation
16         const bool validateTriangle();
17
18         double computePerimeter();
19         double computeArea();
20         std::string determineAngleType();
21     };
22
23     int main(){
24
25         //driver code
26         Triangle set1(40, 30, 110);
27         if(set1.validateTriangle()){
28             std::cout << "The shape is a valid triangle" << std::endl;
29
30             std::cout << "Perimeter (scaled): " << set1.computePerimeter() << std::endl;
31             std::cout << "Area (scaled): " << set1.computeArea() << std::endl;
32             std::cout << "Triangle Type: " << set1.determineAngleType() << std::endl;
33         }else{
34             std::cout << "The shape is NOT a valid triangle. \n";
35         }
36
37         return 0;
```

```
39
40 Triangle:: Triangle(double A, double B, double C){
41     angleA = A;
42     angleB = B;
43     angleC = C;
44     totalAngle = A+B+C;
45 }
46
47 const bool Triangle::validateTriangle() {
48     return (totalAngle == 180 && angleA > 0 && angleB > 0 && angleC > 0);
49 }
50
51 double Triangle::computePerimeter() {
52     double radA = angleA * M_PI / 180.0;
53     double radB = angleB * M_PI / 180.0;
54     double radC = angleC * M_PI / 180.0;
55
56     double sideA = std::sin(radA);
57     double sideB = std::sin(radB);
58     double sideC = std::sin(radC);
59
60     return sideA + sideB + sideC;
61 }
62
63 double Triangle::computeArea() {
64     double radA = angleA * M_PI / 180.0;
65     double radB = angleB * M_PI / 180.0;
66     double radC = angleC * M_PI / 180.0;
67
68     double sideA = std::sin(radA);
69     double sideB = std::sin(radB);
70
71     return 0.5 * sideA * sideB * std::sin(radC);
72 }
73
74 std::string Triangle::determineAngleType() {
75     if (angleA > 90 || angleB > 90 || angleC > 90) {
76         return "Obtuse-angled";
77     }
78     else if (angleA == 90 || angleB == 90 || angleC == 90) {
79         return "Others (Right-angled)";
80     }
81     else {
82         return "Acute-angled";
83     }
84 }
```

OUTPUT:

```
PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE
1\CPE 010-CPE21S1\" ; if ($?) { g++ class_triangle.cpp
The shape is a valid triangle
Perimeter (scaled): 2.08248
Area (scaled): 0.151006
Triangle Type: Obtuse-angled
PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE
```

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini, Claude

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Grammar correction, formal sentence & words, formula equation, code assistance, explanation of concepts.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Was use to provide clear understanding of the "idea" of the program, and to verify if the code is align to the task provided.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.